

RESUMO GERAL DO PROJETO - CAD RIBEIRÃO

Sistema de Gestão Socioassistencial

Versão: 2.0

Última atualização: 28/04/2026

ÍNDICE

1. [Visão Geral](#)
 2. [Stack Tecnológica](#)
 3. [Status de Implementação](#)
 4. [Entidades Principais](#)
 5. [Segurança](#)
 6. [Padrões Arquiteturais](#)
 7. [Regras de Negócio](#)
 8. [Próximos Módulos](#)
 9. [Banco de Dados](#)
 10. [Como Executar](#)
 11. [Próximos Passos](#)
-

VISÃO GERAL

Sistema de **Gestão Socioassistencial** desenvolvido para acompanhamento de famílias e pessoas em situação de vulnerabilidade social, seguindo os princípios do **SUAS (Sistema Único de Assistência Social)**.

O sistema permite:

- Cadastro unificado de famílias e pessoas
 - Gestão de equipamentos socioassistenciais (CRAS, CREAS, SCFV, etc.)
 - Prontuários e atendimentos
 - Encaminhamentos entre equipamentos
 - Vínculos com programas sociais e serviços
 - Auditoria completa de todas as operações
-

STACK TECNOLÓGICA

BACKEND

Framework e Linguagem:

- Spring Boot 3.5.12
- Java 21

Banco de Dados:

- MySQL 8.0
- Flyway (Migrations)

Persistência:

- JPA/Hibernate
- Spring Data JPA

Segurança:

- Spring Security
- JWT (Auth0 java-jwt 4.4.0)
- BCrypt (senhas)

Validação:

- Bean Validation (jakarta.validation)

Utilitários:

- Lombok
- Logback (logging)

Dependências principais (pom.xml):

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
</dependencies>
```

```
</dependencies>
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>
</dependencies>
```

FRONTEND

Framework e Linguagem:

- Next.js 16.2.2
- React 19.2.4
- TypeScript 5

Gerenciamento de Estado:

- TanStack Query (React Query) 5.99.2

Formulários:

- React Hook Form 7.72.1
- Zod 4.3.6 (validação de schemas)
- @hookform/resolvers 5.2.2

UI/Componentes:

- Radix UI (@radix-ui/react-*)
- shadcn/ui 4.1.2
- Lucide React 1.8.0 (ícones)

Estilização:

- Tailwind CSS 4
- class-variance-authority 0.7.1
- clsx 2.1.1

- tailwind-merge 3.5.0

HTTP Client:

-Axios 1.15.0

Utilitários:

- date-fns 4.1.0 (manipulação de datas)
- Sonner 2.0.7 (notificações toast)

Dependências principais (package.json):

```
{
  "dependencies": {
    "@hookform/resolvers": "^5.2.2",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-slot": "^1.2.4",
    "@tanstack/react-query": "^5.99.2",
    "axios": "^1.15.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "date-fns": "^4.1.0",
    "lucide-react": "^1.8.0",
    "next": "16.2.2",
    "react": "19.2.4",
    "react-dom": "19.2.4",
    "react-hook-form": "^7.72.1",
    "shadcn": "^4.1.2",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.5.0",
    "zod": "^4.3.6"
  }
}
```

STATUS DE IMPLEMENTAÇÃO

BACKEND - MÓDULOS COMPLETOS

Módulo	Domain	Repository	Service	Controller	Migration	A
Equipamento	✓	✓	✓	✓	✓	✓
Técnico	✓	✓	✓	✓	✓	✓
TécnicoEquipamento	✓	✓	✓	✓	✓	✓
Usuário	✓	✓	✓	✓	✓	✓

Família	✓	✓	✓	✓	✓	✓
Pessoa	✓	✓	✓	✓	✓	✓
Programa Social	✓	✓	✓	✓	✓	✓
VinculoFamiliaPrograma	✓	✓	✓	✓	✓	✓
Serviço	✓	✓	✓	✓	✓	✓
VinculoPessoaServico	✓	✓	✓	✓	✓	✓
Prontuário	✓	✓	✓	✓	✓	✓
Atendimento	✓	✓	✓	✓	✓	✓
Encaminhamento	✓	✓	✓	✓	✓	✓
AuditLog	✓	✓	✓	—	✓	—

FRONTEND - CRUD IMPLEMENTADOS

Módulo	Listagem	Criação	Edição	Exclusão	Status
Equipamento	✓	✓	✓	✓	Completo
Técnico	✓	✓	✓	✓	Completo
Usuário	✓	✓	✓	✓	Completo
Programa Social	✓	✓	✓	✓	Completo
Serviço	✓	✓	✓	✓	Completo
Família/Pessoa	✓	✓	✓	✓	Completo
Prontuário	🔄	🔄	🔄	—	Em desenvolvimento
Atendimento	🔄	🔄	—	—	Em desenvolvimento

Legenda:

- ✓ Implementado e funcionando
- 🔄 Em desenvolvimento
- Não aplicável

ENTIDADES PRINCIPAIS

1. FAMÍLIA

Descrição: Núcleo familiar cadastrado no sistema. Pode estar vinculada a múltiplos equipamentos via prontuários.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único (AUTO_INCREMENT)
codigo_cadunico	VARCHAR(255)	✗	Código de registro no CadÚnico
situacao	ENUM	✓	ATIVA, INATIVA, SUSPENSA, DESLIGADA
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com Pessoa
- 1:N com Prontuario
- 1:N com Encaminhamento
- 1:N com VinculoFamiliaPrograma

Regras:

- Situação padrão: ATIVA
- Apenas uma pessoa pode ser referência (is_referencia = true)
- Histórico preservado mesmo após inativação

2. PESSOA

Descrição: Membro de uma família. Pode ser a pessoa de referência ou dependente.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
familia_id	BIGINT	✓	FK para Familia
nome	VARCHAR(300)	✓	Nome completo
cpf	VARCHAR(14)	✓	CPE (único quando informado)

cpi	VARCHAR(14)	^	CPF (único quando informado)
nis	VARCHAR	×	Número de Identificação Social
rg_numero	VARCHAR	×	Número do RG
rg_orgao_expeditor	VARCHAR	×	Órgão expeditor do RG
rg_data_expedicao	DATE	×	Data de expedição do RG
data_nascimento	DATE	✓	Data de nascimento
is_referencia	BOOLEAN	✓	Se é a pessoa de referência da família
parentesco	ENUM	×	CONJUGE, FILHO, PAI_MAE, OUTRO
telefone	VARCHAR	×	Telefone de contato
sexo	ENUM	×	MASCULINO, FEMININO
renda_mensal	BIGINT	×	Renda mensal em centavos
logradouro	VARCHAR	×	Endereço - Logradouro
numero_endereco	VARCHAR	×	Endereço - Número
bairro	VARCHAR	×	Endereço - Bairro
cidade	VARCHAR	×	Endereço - Cidade
uf	VARCHAR(2)	×	Endereço - UF
cep	VARCHAR(9)	×	Endereço - CEP
ponto_referencia	VARCHAR	×	Ponto de referência
localizacao_domicilio	ENUM	×	ZONA_URBANA, ZONA_RURAL, ABRIGO
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	×	Data da última atualização
atualizado_por	BIGINT	×	Usuário que atualizou

Relacionamentos:

- N:1 com Família
- 1:N com Atendimento (opcional)
- 1:N com VinculoPessoaServico

Regras:

- CPF único quando informado
- Se is_referencia = true, parentesco deve ser null
- Pessoa de referência deve ter endereço preenchido
- Idade calculada dinamicamente a partir de data_nascimento

3. EQUIPAMENTO

Descrição: Unidade física onde são prestados os serviços socioassistenciais.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
nome	VARCHAR(300)	✓	Nome do equipamento
tipo	ENUM	✓	CRAS, CREAS, SCFV, ACOLHIMENTO, OUTRO
cep	VARCHAR(9)	✗	CEP do equipamento
logradouro	VARCHAR	✗	Endereço - Logradouro
numero	VARCHAR(20)	✗	Endereço - Número
complemento	VARCHAR	✗	Endereço - Complemento
bairro	VARCHAR	✗	Endereço - Bairro
cidade	VARCHAR	✗	Endereço - Cidade
estado	VARCHAR(2)	✗	Endereço - UF
telefone	VARCHAR(20)	✗	Telefone de contato
email	VARCHAR	✗	E-mail do equipamento
ativo	BOOLEAN	✓	Se está em operação
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com Servico
- 1:N com Prontuario
- N:N com Tecnico (via TecnicoEquipamento)
- 1:N com Encaminhamento (origem e destino)

Tipos de Equipamento:

- CRAS: Centro de Referência de Assistência Social (proteção básica)
- CREAS: Centro de Referência Especializado (proteção especial média complexidade)
- SCFV: Serviço de Convivência e Fortalecimento de Vínculos
- ACOLHIMENTO: Abrigos e casas de acolhimento (alta complexidade)
- OUTRO: Outros tipos de equipamentos

4. TÉCNICO

Descrição: Profissional que atua nos equipamentos (assistente social, psicólogo, etc.).

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
nome	VARCHAR(300)	✓	Nome completo
cpf	VARCHAR(14)	✓	CPF (único)
registro_profissional	VARCHAR	✗	CRESS, CRP, etc.
especialidade	ENUM	✓	ASSISTENTE_SOCIAL, PSICOLOGO, OUTRO
ativo	BOOLEAN	✓	Se está em atividade
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com Atendimento
- 1:N com Prontuario
- 1:N com Atendimento
- N:N com Equipamento (via TecnicoEquipamento)
- 1:N com Prontuario

- 1:N com Atendimento
- 1:1 com Usuario
- N:N com Equipamento (via TecnicoEquipamento)
- 1:N com Prontuario

Regras:

- CPF único
- Pode estar vinculado a múltiplos equipamentos simultaneamente

5. TECNICO_EQUIPAMENTO

Descrição: Tabela associativa que registra os vínculos entre técnicos e equipamentos.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
tecnico_id	BIGINT	✓	FK para Tecnico
equipamento_id	BIGINT	✓	FK para Equipamento
data_inicio	DATE	✓	Início do vínculo
data_fim	DATE	✗	Fim do vínculo (null = ativo)
ativo	BOOLEAN	✓	Se o vínculo está ativo
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou

Regras:

- Constraint única: (tecnico_id, equipamento_id, ativo)
- Um técnico pode ter múltiplos vínculos ativos
- Um técnico pode ter múltiplos vínculos ativos
- data_fim = null indica vínculo ativo
- data_fim = null indica vínculo ativo

6. USUÁRIO

Descrição: Credenciais de acesso ao sistema. Todo usuário está vinculado a um técnico.

Campos:

Campo	Tipo	Obrigatório	Descrição
-------	------	-------------	-----------

id	BIGINT	✓	Identificador único
tecnico_id	BIGINT	✓	FK para Tecnico (único)
nome_usuario	VARCHAR	✓	Username para login (único)
senha	VARCHAR	✓	Senha criptografada (BCrypt)
perfil	ENUM	✓	ADMIN, USUARIO
ativo	BOOLEAN	✓	Se pode acessar o sistema
ultimo_acesso	DATETIME	✗	Data/hora do último login
criado_em	DATETIME	✓	Data de cadastro
atualizado_em	DATETIME	✗	Data da última atualização

Relacionamentos:

- 1:1 com Tecnico
- 1:N com AuditLog

Regras:

- JWT gerado no login
- Relacionamento 1:1 com Técnico
- Senhas armazenadas com BCrypt
- Username único

7. SERVIÇO

Descrição: Atividades/grupos oferecidos pelos equipamentos (ex: SCFV, grupos de convivência).

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
equipamento_id	BIGINT	✓	FK para Equipamento
nome	VARCHAR(300)	✓	Nome do serviço
descricao	TEXT	✗	Descrição detalhada
publico_alvo	VARCHAR(300)	✗	Ex: "crianças de 6 a 15 anos"
faixa_etaria_min	INTEGER	✗	Idade mínima

faixa_etaria_max	INTEGER	✗	Idade máxima
dia_semana	VARCHAR(100)	✗	Ex: "segunda, quarta"
horario	VARCHAR(100)	✗	Ex: "14h às 16h"
ativo	BOOLEAN	✓	Se está em funcionamento
criado_em	TIMESTAMP	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	TIMESTAMP	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com Atendimento (quando modalidade = GRUPO)
- 1:N com VinculoPessoaServico
- N:1 com Equipamento

Constraints:

- CHECK (faixa_etaria_min <= faixa_etaria_max)
- Unique: (nome, equipamento_id)

8. VINCULO_PESSOA_SERVICO

Descrição: Registra a participação de pessoas em serviços/grupos.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
pessoa_id	BIGINT	✓	FK para Pessoa
servico_id	BIGINT	✓	FK para Servico
data_entrada	DATE	✓	Início da participação
data_saida	DATE	✗	Fim da participação
status	ENUM	✓	ATIVO, SUSPENSO, CANCELADO
motivo_saida	TEXT	✗	Razão do desligamento
criado_em	TIMESTAMP	✓	Data de cadastro
criado por	BIGINT	✓	Usuário que cadastrou

atualizado_em	TIMESTAMP	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Constraints:

- Unique: (pessoa_id, servico_id, status)

Regras:

- data_entrada padrão = data atual
- Status padrão = ATIVO
- Validação de faixa etária (se definida no serviço)

9. PROGRAMA SOCIAL

Descrição: Programas de transferência de renda e benefícios (ex: Bolsa Família, BPC).

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
nome	VARCHAR(300)	✓	Nome do programa
criterios	TEXT	✗	Descrição dos critérios de elegibilidade
orgao_gestor	VARCHAR	✗	Órgão responsável pelo programa
ativo	BOOLEAN	✓	Se o programa está vigente
criado_em	TIMESTAMP	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	TIMESTAMP	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com VinculoFamiliaPrograma
- 1:N com Atendimento

10. VINCULO_FAMILIA_PROGRAMA

Descrição: Registra a participação de famílias em programas sociais.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
familia_id	BIGINT	✓	FK para Família
programa_id	BIGINT	✓	FK para ProgramaSocial
data_entrada	DATE	✓	Quando entrou no programa
data_saida	DATE	✗	Quando saiu
status	ENUM	✓	ATIVO, SUSPENSO, CANCELADO
motivo_saida	TEXT	✗	Razão da saída
criado_em	TIMESTAMP	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	TIMESTAMP	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Constraints:

- Unique: (familia_id, programa_id)

11. PRONTUÁRIO

Descrição: Registro de acompanhamento de uma família em um equipamento específico. Uma família pode ter múltiplos prontuários (em equipamentos diferentes).

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
familia_id	BIGINT	✓	FK para Família
equipamento_id	BIGINT	✓	FK para Equipamento
tecnico_id	BIGINT	✓	FK para Tecnico (responsável)
data_abertura	DATE	✓	Quando foi aberto
data_fechamento	DATE	✗	Quando foi encerrado
status	ENUM	✓	ABERTO, ENCERRADO, SUSPENSO
motivo_encerramento	TEXT	✗	Razão do encerramento
criado em	DATETIME	✓	Data de cadastro

criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- 1:N com Atendimento
- N:1 com Tecnico
- N:1 com Equipamento
- N:1 com Familia

Regras:

- Status padrão: ABERTO
- Uma família pode ter prontuários simultâneos em diferentes equipamentos
- Histórico nunca é deletado

12. ATENDIMENTO

Descrição: Registro de cada atendimento/contato realizado. Pode ser individual (pessoa específica), familiar (toda a família), relacionado a um serviço ou a um programa social.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
prontuario_id	BIGINT	✓	FK para Prontuario
tecnico_id	BIGINT	✓	FK para Tecnico (quem realizou)
pessoa_id	BIGINT	✗	FK para Pessoa (null = atendimento familiar)
servico_id	BIGINT	✗	FK para Servico (atendimento em grupo)
programa_id	BIGINT	✗	FK para ProgramaSocial (orientações, etc.)
data	DATETIME	✓	Data/hora do atendimento
tipo	ENUM	✓	VISITA_DOMICILIAR, ATENDIMENTO_PRESENCIAL, CONTATO_TELEFONICO, OUTRO
modalidade	ENUM	✓	INDIVIDUAL, GRUPO
descricao	TEXT	✓	Registro detalhado do atendimento

criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- N:1 com ProgramaSocial (opcional)
- N:1 com Servico (opcional)
- N:1 com Pessoa (opcional)
- N:1 com Tecnico
- N:1 com Prontuario

Regras:

- Se modalidade = GRUPO, servico_id pode ser preenchido
- pessoa_id = null indica atendimento à família toda
- Todos os atendimentos são auditados

Exemplos de uso:

- Atendimento individual à criança → pessoa_id preenchido
- Atendimento familiar → pessoa_id = null
- Atividade em grupo SCFV → servico_id preenchido
- Orientação sobre Bolsa Família → programa_id preenchido

13. ENCAMINHAMENTO

Descrição: Registra o encaminhamento de uma família de um equipamento para outro.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
familia_id	BIGINT	✓	FK para Familia
equipamento_origem_id	BIGINT	✓	FK para Equipamento (origem)
equipamento_destino_id	BIGINT	✓	FK para Equipamento (destino)
tecnico_id	BIGINT	✓	FK para Tecnico (quem encaminhou)

			encaminhado,
data	DATETIME	✓	Data do encaminhamento
motivo	TEXT	✓	Justificativa
status	ENUM	✓	PENDENTE, ACEITO, RECUSADO, CONCLUIDO
criado_em	DATETIME	✓	Data de cadastro
criado_por	BIGINT	✓	Usuário que cadastrou
atualizado_em	DATETIME	✗	Data da última atualização
atualizado_por	BIGINT	✗	Usuário que atualizou

Relacionamentos:

- N:1 com Tecnico
- N:1 com Equipamento (destino)
- N:1 com Equipamento (origem)
- N:1 com Familia

Fluxo:

1. Técnico cria encaminhamento (status = PENDENTE)
2. Equipamento destino aceita/recusa
3. Se aceito, pode evoluir para CONCLUIDO
4. AUDIT_LOG

Descrição: Tabela de auditoria que registra todas as operações (INSERT, UPDATE, DELETE) realizadas no sistema.

Campos:

Campo	Tipo	Obrigatório	Descrição
id	BIGINT	✓	Identificador único
tabela	VARCHAR(100)	✓	Nome da tabela afetada
registro_id	BIGINT	✓	ID do registro afetado
acao	ENUM	✓	INSERT, UPDATE, DELETE
dados_antes	JSON	✗	Snapshot anterior (null em INSERT)

dados_depois	JSON	✗	Snapshot novo (null em DELETE)
usuario_id	BIGINT	✓	FK para Usuario (quem realizou)
feito_em	DATETIME	✓	Timestamp da ação

Relacionamentos:

- N:1 com Usuario

Regras:

- Registro automático via AuditService
- Snapshots em formato JSON para flexibilidade
- Nunca é deletado

SEGURANÇA

AUTENTICAÇÃO

JWT (JSON Web Token):

- Biblioteca: Auth0 java-jwt 4.4.0
- Token gerado no login com claims personalizados
- Refresh token implementado

Senha:

- Criptografia: BCrypt
- Nunca armazenada em texto plano

Filtro de Segurança:

- FiltroTokenAcesso: intercepta requisições e valida JWT
- Anotação customizada @UsuarioLogado para injetar ID do usuário autenticado

Endpoints públicos:

- POST /auth/login
- POST /auth/refresh

Exemplo de login:

```
POST /auth/login
{
  "username": "maria.silva",
  "password": "senha123"
}
```

```
}

Response:
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs... ",
  "usuario": {
    "id": 1,
    "username": "maria.silva",
    "perfil": "USUARIO",
    "tecnico": { ... }
  }
}
```

AUTORIZAÇÃO

Perfis:

- ADMIN: Acesso total ao sistema
- USUARIO: Acesso limitado (conforme regras de negócio)

Spring Security:

- Configuração customizada em SecurityConfiguration
- Filtros encadeados
- CORS configurado

Planejado (próximas versões):

- Permissões granulares por módulo
- Perfis adicionais (ex: GESTOR, COORDENADOR)

AUDITORIA

AuditService:

- Método centralizado: registrar(tabela, registroId, acao, dadosAntes, dadosDepois, usuario)
- Invocado automaticamente nos Services após INSERT/UPDATE/DELETE

Exemplo:

```
// Captura estado ANTES
AtendimentoRespostaDTO estadoAnterior = AtendimentoRespostaDTO.fromEntity(atendimento);

// Executa alteração
atendimento.setDescricao("Nova descrição");
Atendimento salvo = repository.save(atendimento);

// Captura estado DEPOIS
```

```

AtendimentoRespostaDTO estadoNovo = AtendimentoRespostaDTO.fromEntity(salvo);

// Registra auditoria
auditService.registrar(
    "atendimento",
    salvo.getId(),
    AcaoAudit.UPDATE,
    estadoAnterior,
    estadoNovo,
    usuario
);

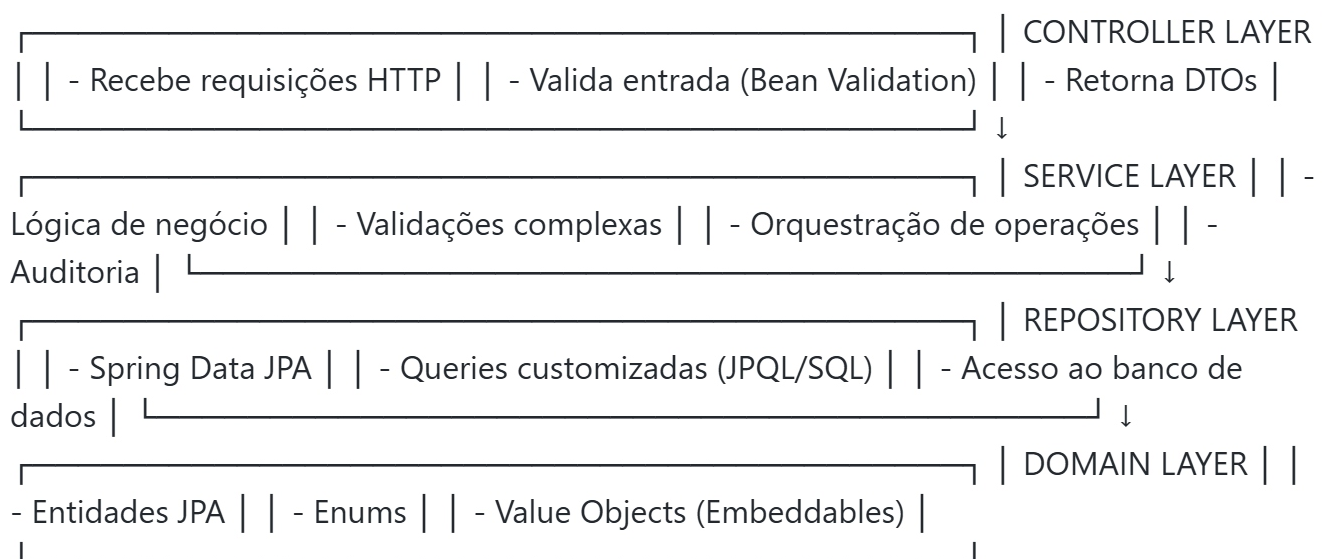
```

Benefícios:

- Rastreabilidade completa
- Conformidade com LGPD
- Histórico imutável

PADRÕES ARQUITETURAIS

ARQUITETURA EM CAMADAS



DTOs (DATA TRANSFER OBJECTS)

Convenções adotadas:

1. Request DTOs (Entrada):

- Sufixo: CadastroDTO, AtualizacaoDTO, CreateRequest, UpdateRequest
- Validações via Bean Validation
- Exemplos: AtendimentoCadastroDTO, ProgramaSocialCreateRequest, UsuarioAtualizacaoDTO

2. Response DTOs (Saída):

- Sufixo: RespostaDTO, Response, DTO
- Método estático fromEntity(Entity)
- Exemplos: AtendimentoRespostaDTO, FamiliaDTO, ServicoResponse

3. Resumo DTOs (Listas/Combos):

- Sufixo: ResumoDTO
- Apenas campos essenciais
- Exemplos: TecnicoResumoDTO (id, nome, especialidade)

Tipos:

- Records (Java 14+): para DTOs imutáveis
- Classes com Lombok: para DTOs mutáveis

Exemplo (Record):

```
public record AtendimentoCadastroDTO(  
    @NotNull Long prontuarioId,  
    Long pessoaId,  
    Long servicoId,  
    Long programaId,  
    @NotNull LocalDateTime data,  
    @NotNull TipoAtendimento tipo,  
    @NotNull ModalidadeAtendimento modalidade,  
    @NotBlank String descricao  
) {}
```

Exemplo (Classe com Lombok):

```
@Getter  
@Setter  
public class ServicoUpdateRequest {  
    private Long equipamentoId;  
  
    @Size(max = 300)  
    private String nome;  
  
    private String descricao;  
  
    @Min(0)  
    private Integer faixaEtariaMin;  
  
    @Min(0)  
    private Integer faixaEtariaMax;  
  
    // ...  
}
```

```
private boolean ativo;  
}
```

VALIDAÇÕES

Bean Validation (Jakarta):

```
@NotNull(message = "Campo obrigatório")  
@NotBlank(message = "Campo não pode ser vazio")  
@Size(min = 3, max = 300, message = "Tamanho inválido")  
@Min(value = 0, message = "Valor mínimo: 0")  
@Email(message = "E-mail inválido")  
@Pattern(regex = "...", message = "Formato inválido")
```

Validações Customizadas (Service Layer):

```
// Exemplo: validar se técnico está vinculado ao equipamento  
boolean vinculado = tecnicoEquipamentoRepository  
    .existsByTecnicoIdAndEquipamentoIdAndAtivoTrue(tecnicoId, equipamentoId);  
  
if (!vinculado) {  
    throw new IllegalStateException("Técnico não vinculado ao equipamento");  
}
```

CONVENÇÕES DE CÓDIGO

Nomenclatura:

- Entidades: PascalCase, singular (ex: Familia, Pessoa)
- Tabelas: snake_case, singular (ex: familia, pessoa)
- Repositories: [Entidade]Repository (ex: FamiliaRepository)
- Services: [Entidade]Service (ex: FamiliaService)
- Controllers: [Entidade]Controller (ex: FamiliaController)

Métodos:

- CRUD básico: cadastrar, buscarPorId, listar, atualizar, deletar
- Queries: buscarPor..., listarPor..., existe...

Exceções:

- EntityNotFoundException (JPA)
- IllegalStateException (regras de negócio)
- IllegalArgumentException (validação de parâmetros)
- Exceções customizadas: [Entidade]NaoEncontradoException

MIGRATIONS (FLYWAY)

Convenção de nomenclatura: V[número]__[descrição].sql

Exemplos: V1__criar_tabela_equipamento.sql V2__criar_tabela_tecnico.sql
V12__adicionar_servico_e_programa_em_atendimento.sql

Regras:

- Nunca alterar migrations já aplicadas
-  Sempre criar nova migration para mudanças
-  Usar sintaxe MySQL (não PostgreSQL)
-  Definir constraints e índices
-  Testar antes de commitar

Estrutura padrão:

```
-- V[N]__[descrição].sql

CREATE TABLE IF NOT EXISTS nome_tabela (
  id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  campo VARCHAR(255) NOT NULL,
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  criado_por BIGINT NOT NULL
);

CREATE INDEX idx_nome_tabela_campo ON nome_tabela(campo);

ALTER TABLE nome_tabela
ADD CONSTRAINT fk_nome_tabela_outra
  FOREIGN KEY (outra_id) REFERENCES outra_tabela(id);
```

REGRAS DE NEGÓCIO IMPLEMENTADAS

#	Regra	Status	Observação
1	Apenas uma pessoa de referência por família		Validado no Service
2	Pessoa de referência deve ter endereço		Validado no Service
3	Parentesco é null quando is_referencia = true		Validado no Service
4	CPF único quando informado		Constraint UNIQUE no banco
5	Técnico vinculado a múltiplos equipamentos		Tabela TecnicoEquipamento

6	Histórico nunca é apagado	✓	Soft delete via status
7	Família pode ter prontuários em múltiplos equipamentos	✓	Sem constraint UNIQUE
8	Atendimento individual pode ter pessoa_id	✓	Campo opcional
9	Atendimento em grupo pode ter servico_id	✓	Campo opcional
10	Atendimento relacionado a programa pode ter programa_id	✓	Campo opcional
11	Validação de faixa etária em VinculoPessoaServico	✓	Validado no Service
12	Não permitir vínculo duplicado ativo	✓	Constraint UNIQUE
13	Todas as ações são auditadas	✓	AuditService centralizado
14	Senhas nunca são salvas em texto puro	✓	BCrypt
15	Todo usuário vinculado a um técnico	✓	FK obrigatória + UNIQUE
16	Prontuário só pode ser criado por técnico vinculado ao equipamento	✓	Validado no Service
17	Atendimento só pode ser registrado em prontuário ABERTO	✓	Validado no Service
18	faixa_etaria_min <= faixa_etaria_max	✓	Constraint CHECK no banco

PRÓXIMOS MÓDULOS

PENDENTES (Backend + Frontend)

1. RELATÓRIOS

Módulos planejados:

Estatísticas:

- Famílias atendidas por equipamento/período
- Atendimentos por técnico/tipo/modalidade
- Programas sociais: famílias vinculadas, desligamentos
- Serviços: frequência, faixa etária predominante

Tecnologias:

- Backend: Queries customizadas, agregações
- Frontend: Gráficos (Chart.js ou Recharts)
- Export: PDF (JasperReports - fase 5), Excel

2. PERMISSÕES GRANULARES

Objetivo: Sistema de permissões mais detalhado que ADMIN/USUARIO.

Perfis planejados:

- ADMIN: Acesso total
- GESTOR: Visualização de relatórios, configurações
- COORDENADOR: Gestão de equipe, validação de encaminhamentos
- TECNICO: Atendimentos, prontuários (apenas seu equipamento)
- CONSULTA: Apenas leitura

Permissões por módulo:

- equipamento:criar, equipamento:editar, equipamento:deletar
- familia:criar, familia:editar, familia:deletar
- atendimento:criar, atendimento:editar
- prontuario:criar, prontuario:encerrar

Implementação:

- Tabelas: perfil, permissao, perfil_permissao
- Anotações customizadas: @RequirePermission("atendimento:criar")

BANCO DE DADOS

CONFIGURAÇÃO

MySQL 8.0:

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/cad_familias
spring.datasource.username=root
spring.datasource.password=sua_senha
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# Flyway
spring.flyway.enabled=true
```

```
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=true
```

MIGRATIONS APLICADAS

Versão	Descrição	Status
V1	Criar tabela equipamento	✓
V2	Criar tabela tecnico	✓
V3	Criar tabela tecnico_equipamento	✓
V4	Criar tabela usuario	✓
V5	Criar tabela familia	✓
V6	Criar tabela pessoa	✓
V7	Criar tabela prontuario	✓
V8	Criar tabela atendimento	✓
V9	Criar tabela encaminhamento	✓
V10	Criar tabela servicos	✓
V11	Criar tabela vinculo_pessoa_servico	✓
V12	Adicionar servico_id e programa_id em atendimento	✓
V13	Criar tabela programa_social	✓
V14	Criar tabela vinculo_familia_programa	✓
V15	Criar tabela audit_log	✓

ÍNDICES CRIADOS

Performance otimizada para:

- Buscas por CPF (pessoa)
- Buscas por família
- Listagens de atendimentos por prontuário
- Listagens de prontuários por equipamento/técnico
- Vínculos ativos (servico/programa)
- Encaminhamentos por status
- Exemplos:

```
CREATE INDEX idx_pessoa_cpf ON pessoa(cpf);
CREATE INDEX idx_atendimento_prontuario ON atendimento(prontuario_id);
CREATE INDEX idx_vinculo_pessoa_servico_status ON vinculo_pessoa_servico(status);
CREATE INDEX idx_servicos Equipamento_id ON servicos(Equipamento_id);
```

CONSTRAINTS

Foreign Keys:

- ON DELETE CASCADE: quando apropriado (ex: pessoa → família)
- ON DELETE SET NULL: para relacionamentos opcionais (ex: atendimento → serviço)
- ON DELETE RESTRICT: padrão (impede deleção se houver dependentes)

```
-- CPF único
ALTER TABLE pessoa ADD CONSTRAINT uq_pessoa_cpf UNIQUE (cpf);

-- Técnico-Equipamento ativo único
ALTER TABLE tecnico_equipamento
ADD CONSTRAINT uk_tecnico_equipamento_ativo
UNIQUE (tecnico_id, equipamento_id, ativo);

-- Vínculo família-programa único
ALTER TABLE vinculo_familia_programa
ADD CONSTRAINT uk_familia_programa_ativo
UNIQUE (familia_id, programa_id);
```

Check Constraints:

```
-- Faixa etária lógica
ALTER TABLE servicos
ADD CONSTRAINT ck_servico_faixa_etaria
CHECK (faixa_etaria_min IS NULL OR faixa_etaria_max IS NULL
      OR faixa_etaria_min <= faixa_etaria_max);
```

BACKUP E MANUTENÇÃO

Recomendações:

Backup diário automatizado

- Retenção de 30 dias
- Teste de restore semanal
- Monitoramento de espaço em disco

- Análise de slow queries

Script exemplo (mysqldump):

```
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
mysqldump -u root -p cad_familias > backup_${DATE}.sql
```

COMO EXECUTAR

REQUISITOS

Backend:

- Java 21 ou superior
- Maven 3.8+
- MySQL 8.0+

Frontend:

- Node.js 18+ ou 20+
- npm ou yarn

BACKEND

1. Configurar banco de dados:

```
CREATE DATABASE cad_familias CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

2. Configurar application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/cad_familias
spring.datasource.username=root
spring.datasource.password=sua_senha_aqui
```

3. Executar:

```
cd backend
mvn clean install
mvn spring-boot:run
```

Backend rodando em: <http://localhost:8080>

Endpoints principais:

- POST `api/atendimentos`
- GET `api/equipamentos`
- POST `api/familias`
- GET `api/familias`
- POST `auth/refresh`
- POST `auth/login`

FRONTEND

1. Instalar dependências:

```
cd frontend
npm install
```

2. Configurar variáveis de ambiente:

```
# .env.local
NEXT_PUBLIC_API_URL=http://localhost:8080
```

3. Executar:

```
npm run dev
```

Frontend rodando em: <http://localhost:3000>

PRIMEIRO ACESSO

Criar usuário administrador inicial:

```
-- 1. Criar técnico
INSERT INTO tecnico (nome, cpf, especialidade, ativo, criado_em, criado_por)
VALUES ('Admin Sistema', '00000000000', 'OUTRO', 1, NOW(), 1);

-- 2. Criar usuário (trocar senha depois!)
INSERT INTO usuario (tecnico_id, nome_usuario, senha, perfil, ativo, criado_em)
VALUES (
  1,
  'admin',
  '$2a$10$abcdefg...', -- senha criptografada com BCrypt
  'ADMIN',
  1,
  NOW()
);
```

Gerar senha BCrypt:

```
String senhaCriptografada = new BCryptPasswordEncoder().encode("senha123");
System.out.println(senhaCriptografada);
```

PRÓXIMOS PASSOS

CURTO PRAZO (1-2 meses)

#	Tarefa	Responsável	Status
1	Finalizar Prontuário (frontend)	—	
2	Finalizar Atendimento (frontend)	—	
3	Testes de integração (backend)	—	
4	Documentação com Swagger/OpenAPI	—	
5	Deploy em homologação	—	

MÉDIO PRAZO (3-6 meses)

#	Tarefa	Status
1	Implementar MedidaSocioeducativa (backend + frontend)	
2	Implementar TrabalhoInfantil (backend + frontend)	
3	Módulo de Relatórios (estatísticas básicas)	
4	Permissões granulares (perfis adicionais)	
5	Notificações (alertas de prazos)	
6	Testes automatizados (JUnit + Mockito)	
7	Deploy em produção	

LONGO PRAZO (6-12 meses)

#	Tarefa	Status
1	Integração com CadÚnico federal	
2	Relatórios avançados (gráficos, dashboards)	
3	App mobile (React Native ou Flutter)	

4	Sistema de mensageria (e-mail, SMS)	
5	BI/Analytics	

Repositório

projeto/ ├── backend/ | ├── src/ | ├── pom.xml | └── README.md ├── frontend/ |
 ├── src/ | ├── package.json | └── README.md └── docs/ ├── RESUMO_GERAL.md ←
 Este documento └── DIAGRAMA FR.md └── FLUXOS.md └── API.md